

Research Report

DNS Tunneling & Detection Basics

Objective

- *Understand the core concept and mechanics of DNS tunneling*
- *Identify common attack patterns and adversary techniques*
- *Learn basic to advanced detection approaches using network traffic analysis*

Expected Output

- *A clear explanation of how DNS tunneling bypasses traditional security*
- *Examples of tunneling tools and data manipulation techniques*
- *Actionable detection ideas leveraging Deep Packet Inspection (DPI) and mathematical models*

Tools & Technologies Referenced

- **Concepts:** *Shannon Entropy, Inter-arrival Time Analysis, Deep Packet Inspection (DPI)*
- **Simulated Tools:** *Suricata, Python (Data Preprocessing), DNScat2, Iodine*
- **Literature:** *"DNS Tunneling: Threat Landscape and Improved Detection Solutions", "DNS Tunnelling, Exfiltration and Detection over Cloud Environments", and "Deep Learning Based DNS Tunneling Detection"*

Introduction

In the landscape of modern enterprise cybersecurity, organizations invest heavily in sophisticated perimeter defenses—like Next-Generation Firewalls (NGFW) and Intrusion Prevention Systems (IPS)—to keep malicious actors out. However, even the most fortified networks possess a fundamental "open door" that cannot simply be locked: the Domain Name System (DNS). Acting as the internet's phonebook, DNS is responsible for translating human-readable web addresses into machine-readable IP addresses. Because this continuous translation is vital for almost all digital operations, security appliances are universally configured to trust DNS traffic by default, leaving Port 53 open and largely unmonitored.

Threat actors have recognized this inherent trust as a massive vulnerability. Enter DNS Tunneling - a stealthy and highly effective evasion technique that weaponizes the DNS protocol itself. Instead of attacking a network head-on, adversaries encapsulate stolen data and malicious commands within the structure of seemingly normal DNS queries. To traditional security systems, this traffic appears as harmless, routine web browsing. In reality, it acts as a covert channel, allowing attackers to quietly exfiltrate sensitive data, bypass firewall restrictions, and establish persistent

Command and Control (C&C) connections right under the radar of network administrators.

This report aims to demystify the mechanics behind DNS tunneling. It explores how adversaries manipulate standard protocol rules using common attack tools, and explains exactly why traditional Layer 3/4 defenses fail to stop them. Furthermore, it outlines a robust, multi-dimensional detection architecture. By combining Deep Packet Inspection (DPI) for deep contextual awareness, mathematical models like Shannon Entropy for lexical anomaly detection, and temporal analysis for behavioral tracking, we can effectively identify and neutralize these covert communications without disrupting legitimate business traffic.

Implementation

Explanation of DNS Tunneling

To understand why DNS tunneling is so dangerous, we first must understand the blind spots of traditional network defense.

Normally, the Domain Name System (DNS) acts as the "phonebook" of the internet, translating human-readable domain names (like google.com) into IP addresses. Because this translation is essential for nearly all network activity, traditional Stateful Firewalls are configured to inherently trust and allow traffic flowing through Port 53 (the standard DNS port).

The Blind Spot: Traditional firewalls operate primarily at Layer 3 and Layer 4 of the OSI model. They inspect the "envelope" of the packet—checking the source IP, destination IP, and Port. They do not possess the capability to look inside the actual payload (Layer 7).

The Mechanism of the Attack: When an internal machine is compromised, the malware does not send stolen data directly to the hacker's IP address. Doing so would alert the firewall immediately. Instead, the malware takes advantage of the recursive nature of DNS.

1. The malware hides stolen data inside a DNS query and sends it to a trusted, legitimate local DNS resolver (such as Google's 8.8.8.8 or the ISP's DNS).
2. The firewall sees an internal machine talking to a trusted Google server on Port 53 and allows it to pass.
3. Because the requested domain actually belongs to the attacker (e.g., [stolen_data].badguy.com), the trusted DNS server unwittingly acts as a "courier," forwarding the query across the internet until it reaches the attacker's Authoritative Name Server.

Through this process, the attacker completely bypasses the firewall without ever making a direct connection to the victim.

Common Attack Patterns and Tools

DNS Tunneling is not limited to simply stealing data; it is a full-fledged communication channel. Adversaries typically rely on the following techniques to exploit the protocol:

1. Data Exfiltration via Subdomain Encoding

DNS protocols dictate strict rules for domain names: they can only contain alphanumeric characters (a-z, 0-9) and hyphens, with a maximum length of 63 characters per subdomain. To extract raw data (such as internal documents, passwords, or system logs), the malware must first chunk the data and encode it—most commonly using Base32 or Base64 encoding.

For example, a password string might be encoded and appended as a subdomain: ZGF5LWxhLW1hdC1raGF1.badguy.com. The attacker's server receives this query, strips away the .badguy.com root domain, and decodes the subdomain to retrieve the stolen information.

2. Command and Control (C&C) & Persistence

While data exfiltration is a one-way street, modern attackers use DNS to establish a two-way Command and Control (C&C) channel. To send malicious commands back to the infected machine, attackers abuse specific DNS record types:

- Instead of requesting standard A records (which only return a small IPv4 address), the malware requests TXT or NULL records.
- A TXT record can hold up to 255 characters of arbitrary text. The attacker packs encrypted executable commands into this space, allowing them to remotely control the machine, download secondary payloads (like ransomware), or move laterally within the network.

3. Common Malicious Tools

According to recent threat landscape research, two of the most heavily utilized open-source tools for this purpose are:

- DNScat2: Specifically designed to create an encrypted C&C channel over DNS. It heavily utilizes the "*beaconing*" technique to maintain a persistent connection with the attacker.
- Iodine: A tool built to tunnel entire IPv4 data streams through a DNS server, effectively giving the attacker a full VPN-like connection into the victim's network.

Basic Detection Approaches

Because attackers disguise their traffic as legitimate DNS queries, catching them requires a "Defense in Depth" approach. Security systems must utilize Deep Packet Inspection (DPI) tools, like Suricata, to extract Layer 7 context and feed it into analytical engines.

Based on academic research, detection strategies rely on extracting three core categories of features:

1. Lexical Analysis (Mathematical Anomaly Detection)

When data is encrypted or encoded into Base64, it loses the natural patterns of human language. The resulting subdomains look like random gibberish.

- **Shannon Entropy:** We apply this mathematical formula to measure the randomness (chaos) of the characters in the subdomain. A legitimate domain like `translate.google.com` will have a relatively low entropy score, whereas a malicious payload like `ZGF5LWxhLW1hdC1raGF1` will spike significantly.
- **Mitigating False Positives:** To avoid blocking legitimate but long URLs (e.g., Vietnamese domains like `congtaevien.osbholding.com`), the entropy calculation must be paired with length thresholds. The analytical engine should only flag domains that possess *both* an unusually long subdomain (e.g., > 30 characters) *and* a high entropy score.

2. Protocol Feature Analysis (Contextual Inspection)

Using DPI, the system looks beyond the domain name and inspects the structural intent of the query. Normal web browsing predominantly generates queries for A or AAAA records. If the DPI logs reveal that a specific internal IP address is suddenly requesting an unusually high volume of TXT, CNAME, or NULL records directed at a single root domain, it is a massive indicator of an active C&C channel attempting to download a payload.

3. Traffic and Volume Analysis (Behavioral Tracking)

To bypass length restrictions and entropy filters, highly sophisticated attackers might utilize "Low and Slow" techniques—sending very short strings over a long period. To detect this, we analyze the behavior of the traffic over time:

- **Query Frequency:** Tracking the total volume of queries to a specific domain within a given time window (e.g., rate-limiting to catch a machine asking for the same domain 50 times a minute).
- **Inter-arrival Time Analysis:** Malware relies on automated loops and "sleep" timers. This creates a "beaconing" effect where queries are sent out at perfectly regular intervals (e.g., exactly every 5.0 seconds). By measuring the standard deviation of the time between queries, the system can distinguish

between the random, bursty browsing habits of a human and the robotic, scheduled heartbeat of a malware script.

Practical Implementation – The DNS Entropy Analyzer

To bridge the gap between academic theory and real-world application, a custom Python-based detection tool was developed as part of this research. The objective of this script is to act as a lightweight, highly accurate Rule Engine that processes DNS queries and flags potential exfiltration or C&C channels.

Rather than relying on complex, resource-heavy machine learning models, this program utilizes a deterministic mathematical approach. It is built upon three core operational phases: Data Preprocessing, Mathematical Analysis, and Dual-Threshold Evaluation.

1. Data Preprocessing: Isolating the Threat

In real-world security operations, analyzing an entire domain string (e.g., ZGF5LWxh.badguy.net) often leads to inaccurate calculations because the legitimate root domain and suffix dilute the randomness of the malicious payload.

To solve this, the script integrates the *tlextract* library. This tool intelligently parses and strips away the Top-Level Domain (TLD) and the Root Domain, isolating only the Subdomain. By feeding only the specific "container" of the payload into the mathematical engine, the system ensures maximum accuracy and eliminates noise from the evaluation process.

2. Mathematical Analysis: Calculating Shannon Entropy

At the heart of the script lies the implementation of the Shannon Entropy formula:

$$H(X) = -\sum_{i=1}^k P(x_i) \log_2 P(x_i)$$

Whereas

- *H(X) (The Entropy Score): This is the final calculated value representing the overall randomness of the dataset X (the DNS subdomain string).*
- *k (Total Unique Characters): In the summation algorithm, 'k' represents the total number of distinct or unique characters found in the string.*
- *x_i (The Specific Character): This represents an individual, unique character within the string at index 'i' (for example, the letter 'a', the number '1', or a hyphen '-').*
- *P(x_i) (The Probability): It is calculated by dividing the character's occurrence count by the total length of the string. In natural language (like legitimate*

domain names), characters often repeat, resulting in a higher probability. In an encoded malware payload, characters rarely repeat, resulting in a very low probability.

- $\log_2$ (Base-2 Logarithm): Because computing systems process information in binary (0s and 1s), the base-2 logarithm is used to calculate the "information density" or bit-weight of each character.
- Σ (The Summation): It directs the algorithm to calculate the probability and logarithm for each unique character and then add all those individual results together
- The Negative Sign (-): The negative sign at the very front of the formula simply inverts this result, ensuring that the final Entropy score (H) is a readable, positive number

Using Python's optimized Counter collection, the program evaluates the frequency of each character within the isolated subdomain. If a hacker encodes stolen data using Base64, the resulting string will contain a highly randomized, non-repeating set of characters. The script calculates this chaos and assigns it a definitive numerical score. The higher the score, the more unnatural the string.

3. Dual-Engine Threshold: Decreasing False Positives

The most significant challenge in purely mathematical detection is the occurrence of False Positives. Legitimate domains with unique, non-repeating characters (such as congtaevien.osbholding.com) can inadvertently trigger a high entropy score.

To counteract this, the program does not rely on entropy alone. It implements a Dual-Engine Logic that requires a DNS query to fail two distinct tests simultaneously before an alert is triggered:

- Length Threshold: The subdomain must be unusually long (Length ≥ 15 characters), as attackers need length to exfiltrate meaningful amounts of data.
- Entropy Threshold: The string must exhibit extreme mathematical chaos (Score ≥ 3.5).

By combining these two metrics, the script effectively filters out short, legitimate domains regardless of their entropy, ensuring that alerts are only generated for genuine anomalies.

Note: The developing source code for the detection engine, demonstrating this logic in action, is provided in the file named `dns_entropy_analyzer.py`

Future Work & Next Steps

Currently, the developed Python module successfully performs static lexical analysis using Shannon Entropy and Subdomain Length thresholds. However, to combat 'low and slow' evasion tactics, the system requires temporal analysis. The next phase of

this project involves integrating the script with Deep Packet Inspection (DPI) tools like Suricata. By continuously parsing Suricata's eve.json logs, the system will implement stateful tracking to monitor query frequency and inter-arrival times per IP address. Combining these volume-based features with the existing entropy logic will complete the multi-dimensional detection architecture.

Conclusion

To wrap up this research, one crucial reality becomes clear: in modern cybersecurity, simply locking the "front door" with traditional firewalls is no longer enough. Because the Domain Name System (DNS) is absolutely essential for the internet to function, it is almost always trusted by default. As we have seen, adversaries actively exploit this trust. By weaponizing tools like DNScat2 or Iodine, they can easily slip stolen data and remote commands right past our standard defenses, hiding their activity within the everyday noise of web traffic.

However, this report also proves that we do not have to remain blind to these covert channels. By looking deeper into the traffic—specifically at Layer 7—we can expose the attackers' footprints. The custom Python detection engine developed for this project serves as a practical proof of concept. By applying the Shannon Entropy formula alongside length thresholds, we successfully created a mathematical "filter" that can spot the chaotic, unnatural strings generated by malware. Most importantly, by combining these two rules, the tool acts smartly; it flags genuine threats without raising false alarms over normal, everyday business browsing.